NAME:CHETHRIKASRI KM COURSECODE:CSA5112

REG NO:192511112 SLOT:B

EXPERIMENT 1

IMPLEMENTATION OF DES

AIM

To implement the Data Encryption Standard (DES) algorithm using Python.

ALGORITHM

Step-1: Start the program.

Step-2: Import the DES cipher and padding modules.

Step-3: Read the plaintext message from the user.

Step-4: Define an 8-byte secret key for DES encryption.

Step-5: Create the DES cipher in ECB mode using the secret key.

Step-6: Encrypt the plaintext after applying padding.

Step-7: Decrypt the ciphertext using the same secret key.

Step-8: Remove the padding from the decrypted text.

Step-9: Display the encrypted text and decrypted text.

Step-10: Stop the program.

PROGRAM (Python)

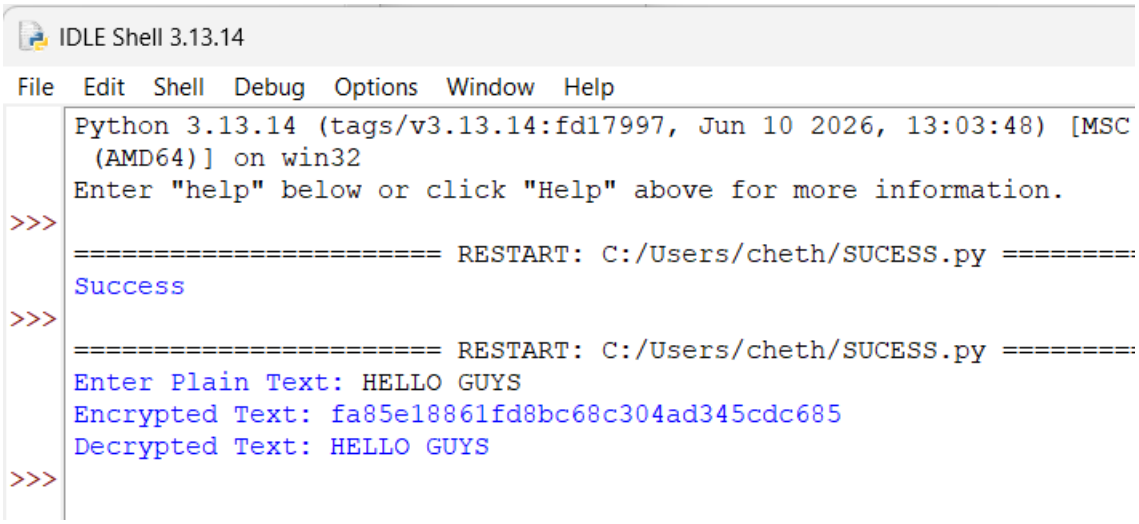
SUCESS.py - C:/Users/cheth/SUCESS.py (3.13.14)

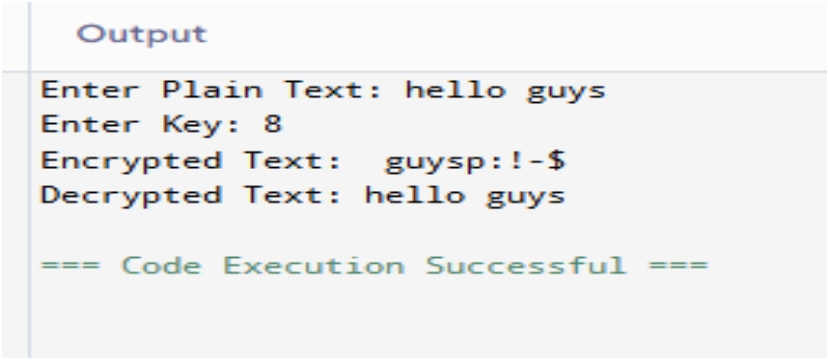
File Edit Format Run Options Window Help

```
from Crypto.Cipher import DES
from Crypto.Util.Padding import pad, unpad

key = b'12345678'

plain = input("Enter Plain Text: ")

cipher = DES.new(key, DES.MODE_ECB)

encrypted = cipher.encrypt(pad(plain.encode(), 8))
print("Encrypted Text:", encrypted.hex())

decrypted = unpad(cipher.decrypt(encrypted), 8)
print("Decrypted Text:", decrypted.decode())
```

```
def feistel_encrypt(text, key, rounds):
    if len(text) % 2 != 0:
        text += " "
    mid = len(text) // 2
    left = text[:mid]
    right = text[mid:]
    for r in range(rounds):
        new_left = right
        new_right = ""
        for i in range(len(left)):
            f = chr((ord(right[i]) ^ key[r % len(key)]) % 256)
            new_right += chr(ord(left[i]) ^ ord(f))
        left = new_left
        right = new_right
    return left + right
def feistel_decrypt(cipher, key, rounds):
    mid = len(cipher) // 2
    left = cipher[:mid]
    right = cipher[mid:]
    for r in range(rounds - 1, -1, -1):
        old_right = left
        old_left = ""
        for i in range(len(right)):
            f = chr((ord(old_right[i]) ^ key[r % len(key)]) % 256)
            old_left += chr(ord(right[i]) ^ ord(f))
        left = old_left
        right = old_right

    return (left + right).rstrip()
text = input("Enter Plain Text: ")
key = input("Enter Key: ")

key = [ord(c) for c in key]

cipher = feistel_encrypt(text, key, 4)
print("Encrypted Text:", cipher)

plain = feistel_decrypt(cipher, key, 4)
print("Decrypted Text:", plain)
```

OUTPUT





RESULT

Thus, the Data Encryption Standard (DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using the secret key.

EXPERIMENT 2

IMPLEMENTATION OF 2DES (DOUBLE DATA ENCRYPTION STANDARD)

AIM

To implement a simplified Double Data Encryption Standard (2DES) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read two encryption keys (Key 1 and Key 2).

Step 4: Encrypt the plaintext using Key 1.

Step 5: Encrypt the obtained ciphertext again using Key 2.

Step 6: Display the encrypted text.

Step 7: Decrypt the encrypted text using Key 2.

Step 8: Decrypt the obtained text using Key 1.

Step 9: Display the original plaintext.

Step 10: Stop the program.

PROGRAM (Python)

```python
def encrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

def decrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

plain = input("Enter Plain Text: ")
key1 = int(input("Enter Key 1: "))
key2 = int(input("Enter Key 2: "))

cipher1 = encrypt(plain, key1)
cipher2 = encrypt(cipher1, key2)

print("Encrypted Text:", cipher2)

temp = decrypt(cipher2, key2)
original = decrypt(temp, key1)

print("Decrypted Text:", original)
```

OUTPUT

```
Output
Enter Plain Text: HELLO EVERYONE
Enter Key 1: 5
Enter Key 2: 9
Encrypted Text: DI@@C,IZI^UCBI
Decrypted Text: HELLO EVERYONE

=== Code Execution Successful ===
```

RESULT

Thus, the simplified Double Data Encryption Standard (2DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using two different keys.

EXPERIMENT 3

IMPLEMENTATION OF 3DES (TRIPLE DATA ENCRYPTION STANDARD)

AIM

To implement a simplified Triple Data Encryption Standard (3DES) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext from the user.

Step 3: Read three encryption keys (Key 1, Key 2, and Key 3).

Step 4: Encrypt the plaintext using Key 1.

Step 5: Decrypt the encrypted text using Key 2.

Step 6: Encrypt the obtained text again using Key 3.

Step 7: Display the encrypted text.

Step 8: Decrypt the ciphertext using Key 3.

Step 9: Encrypt the obtained text using Key 2.

Step 10: Decrypt the obtained text using Key 1 to obtain the original plaintext.

Step 11: Display the decrypted text.

Step 12: Stop the program.

PROGRAM (Python)

```python
def encrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

def decrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

plain = input("Enter Plain Text: ")
key1 = int(input("Enter Key 1: "))
key2 = int(input("Enter Key 2: "))
key3 = int(input("Enter Key 3: "))

# 3DES Encryption (E-D-E)
step1 = encrypt(plain, key1)
step2 = decrypt(step1, key2)
cipher = encrypt(step2, key3)

print("Encrypted Text:", cipher)

# 3DES Decryption
step3 = decrypt(cipher, key3)
step4 = encrypt(step3, key2)
original = decrypt(step4, key1)

print("Decrypted Text:", original)
```

OUTPUT

```
Output

Enter Plain Text: CHETHRIKA
Enter Key 1: 7
Enter Key 2: 6
Enter Key 3: 4
Encrypted Text: FM@QMWLND
Decrypted Text: CHETHRIKA

=== Code Execution Successful ===
```

RESULT

Thus, the simplified Triple Data Encryption Standard (3DES) algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using three different keys.

EXPERIMENT 4(A)

IMPLEMENTATION OF DES – ECB (Electronic Code Book) MODE

AIM

To implement the Electronic Code Book (ECB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext and encryption key from the user.

Step 3: Divide the plaintext into individual blocks.

Step 4: Encrypt each block independently using the key.

Step 5: Display the encrypted text.

Step 6: Decrypt the encrypted text using the same key.

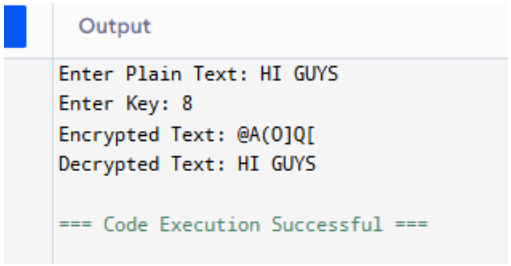
Step 7: Display the original plaintext.

Step 8: Stop the program.

PROGRAM(PYTHON)

```python
def encrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

def decrypt(text, key):
    result = ""
    for ch in text:
        result += chr(ord(ch) ^ key)
    return result

plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))

cipher = encrypt(plain, key)
print("Encrypted Text:", cipher)

text = decrypt(cipher, key)
print("Decrypted Text:", text)
```

OUTPUT



RESULT

Thus, the simplified DES Electronic Code Book (ECB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(B)

IMPLEMENTATION OF DES – CBC (Cipher Block Chaining) MODE

AIM

To implement the Cipher Block Chaining (CBC) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the previous block with the IV.

Step 4: XOR the plaintext character with the previous block.

Step 5: Encrypt the result using the key.

Step 6: Store the encrypted character as the current ciphertext.

Step 7: Use the current ciphertext as the previous block for the next character.

Step 8: Repeat Steps 4–7 until all characters are encrypted.

Step 9: Display the encrypted text.

Step 10: Decrypt the ciphertext using the same key and IV.

Step 11: Display the decrypted text.

Step 12: Stop the program.

PROGRAM(PYTHON)

```python
def encrypt(text, key, iv):
    cipher = ""
    prev = iv

    for ch in text:
        c = (ord(ch) ^ prev) ^ key
        cipher += chr(c)
        prev = c

    return cipher


def decrypt(cipher, key, iv):
    plain = ""
    prev = iv

    for ch in cipher:
        p = (ord(ch) ^ key) ^ prev
        plain += chr(p)
        prev = ord(ch)

    return plain


plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
iv = int(input("Enter IV: "))

cipher = encrypt(plain, key, iv)
print("Encrypted Text:", cipher)

text = decrypt(cipher, key, iv)
print("Decrypted Text:", text)
```

OUTPUT

```
Enter Plain Text: computer
Enter Key: 5
Enter IV: 10
Encrypted Text: l·n·z
Decrypted Text: computer

=== Code Execution Successful ===
```

RESULT

Thus, the simplified DES Cipher Block Chaining (CBC) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(c)

IMPLEMENTATION OF DES – CFB (Cipher Feedback) MODE

AIM

To implement the Cipher Feedback (CFB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the feedback value with the IV.

Step 4: Encrypt the feedback value using the key.

Step 5: XOR the encrypted feedback with the plaintext to generate the ciphertext.

Step 6: Use the generated ciphertext as the new feedback value.

Step 7: Repeat the process until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Decrypt the ciphertext using the same key and IV.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

```python
def encrypt(text, key, iv):
    cipher = ""
    feedback = iv
    for ch in text:
        temp = feedback ^ key
        c = ord(ch) ^ temp
        cipher += chr(c)
        feedback = c

    return cipher
def decrypt(cipher, key, iv):
    plain = ""
    feedback = iv

    for ch in cipher:
        temp = feedback ^ key
        p = ord(ch) ^ temp
        plain += chr(p)
        feedback = ord(ch)
    return plain
plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
iv = int(input("Enter IV: "))

cipher = encrypt(plain, key, iv)

print("Encrypted Text:", cipher)

text = decrypt(cipher, key, iv)

print("Decrypted Text:", text)
```

OUTPUT

```
Output
Enter Plain Text: happy
Enter Key: 5
Enter IV: 7
Encrypted Text: j{r
Decrypted Text: happy

=== Code Execution Successful ===
```

RESULT

Thus, the simplified DES Cipher Feedback (CFB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(D)

IMPLEMENTATION OF DES – OFB (Output Feedback) MODE

AIM

To implement the Output Feedback (OFB) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and Initialization Vector (IV).

Step 3: Initialize the feedback value with the IV.

Step 4: Encrypt the feedback value using the key.

Step 5: XOR the encrypted feedback with the plaintext to generate the ciphertext.

Step 6: Use the encrypted feedback (not the ciphertext) as the feedback for the next round.

Step 7: Repeat the process until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Decrypt the ciphertext using the same key and IV.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

main.py

```python
def encrypt(text, key, iv):
    cipher = ""
    feedback = iv
    for ch in text:
        feedback = feedback ^ key
        c = ord(ch) ^ feedback
        cipher += chr(c)
    return cipher
def decrypt(cipher, key, iv):
    plain = ""
    feedback = iv
    for ch in cipher:
        feedback = feedback ^ key
        p = ord(ch) ^ feedback
        plain += chr(p)
    return plain
plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
iv = int(input("Enter IV: "))

cipher = encrypt(plain, key, iv)
print("Encrypted Text:", cipher)

text = decrypt(cipher, key, iv)
print("Decrypted Text:", text)
```

OUTPUT

Output

```
Enter Plain Text: java program
Enter Key: 6
Enter IV: 7
Encrypted Text: kfwf!wshfu`j
Decrypted Text: java program

=== Code Execution Successful ===
```

RESULT

Thus, the simplified DES Output Feedback (OFB) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 4(E)

IMPLEMENTATION OF DES – OF CTR MODE

AIM

To implement the Counter (CTR) mode of DES using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext, key, and counter value from the user.

Step 3: Initialize the counter.

Step 4: Encrypt the counter using the key.

Step 5: XOR the encrypted counter with the plaintext to generate the ciphertext.

Step 6: Increment the counter for the next character.

Step 7: Repeat Steps 4–6 until all characters are encrypted.

Step 8: Display the encrypted text.

Step 9: Reset the counter and decrypt the ciphertext using the same key.

Step 10: Display the decrypted text.

Step 11: Stop the program.

PROGRAM (Python)

main.py

```python
def encrypt(text, key, counter):
    cipher = ""

    for ch in text:
        c = ord(ch) ^ (counter ^ key)
        cipher += chr(c)
        counter += 1

    return cipher


def decrypt(cipher, key, counter):
    plain = ""

    for ch in cipher:
        p = ord(ch) ^ (counter ^ key)
        plain += chr(p)
        counter += 1

    return plain


plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
counter = int(input("Enter Counter Value: "))

cipher = encrypt(plain, key, counter)
print("Encrypted Text:", cipher)

text = decrypt(cipher, key, counter)
print("Decrypted Text:", text)
```

OUTPUT

Output

```
Enter Plain Text: happy days
Enter Key: 6
Enter Counter Value: 8
Encrypted Text: fn|}s+lhod
Decrypted Text: happy days

=== Code Execution Successful ===
```

RESULT

Thus, the simplified DES Counter (CTR) mode was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 5

IMPLEMENTATION OF RSA ALGORITHM

AIM

To implement the RSA algorithm for encryption and decryption using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read two prime numbers (p and q) from the user.

Step 3: Compute $n = p \times q$.

Step 4: Compute Euler's Totient Function $\varphi(n) = (p - 1) \times (q - 1)$.

Step 5: Choose the public key e such that it is relatively prime to $\varphi(n)$.

Step 6: Compute the private key d.

Step 7: Read the plaintext message.

Step 8: Encrypt the plaintext using the public key.

Step 9: Display the encrypted message.

Step 10: Decrypt the encrypted message using the private key.

Step 11: Display the decrypted message.

Step 12: Stop the program.

PROGRAM (Python)

```python
def gcd(a, b):
    while b != 0:
        a, b = b, a % b
    return a

p = int(input("Enter Prime Number p: "))
q = int(input("Enter Prime Number q: "))

n = p * q
phi = (p - 1) * (q - 1)

e = 2
while e < phi:
    if gcd(e, phi) == 1:
        break
    e += 1

d = 1
while (d * e) % phi != 1:
    d += 1

msg = int(input("Enter Message (number): "))

cipher = (msg ** e) % n
print("Encrypted Message:", cipher)

plain = (cipher ** d) % n
print("Decrypted Message:", plain)
```

OUTPUT

```
Enter Prime Number p: 11
Enter Prime Number q: 13
Enter Message (number): 9
Encrypted Message: 48
Decrypted Message: 9

=== Code Execution Successful ===
```

RESULT

Thus, the RSA algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly.

EXPERIMENT 6

IMPLEMENTATION OF DIFFIE–HELLMAN KEY EXCHANGE ALGORITHM

AIM

To implement the Diffie–Hellman Key Exchange Algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the prime number (P) and primitive root (G).

Step 3: Read the private keys of User A and User B.

Step 4: Compute the public key of User A.

Step 5: Compute the public key of User B.

Step 6: Exchange the public keys.

Step 7: Compute the secret key at User A.

Step 8: Compute the secret key at User B.

Step 9: Display both secret keys.

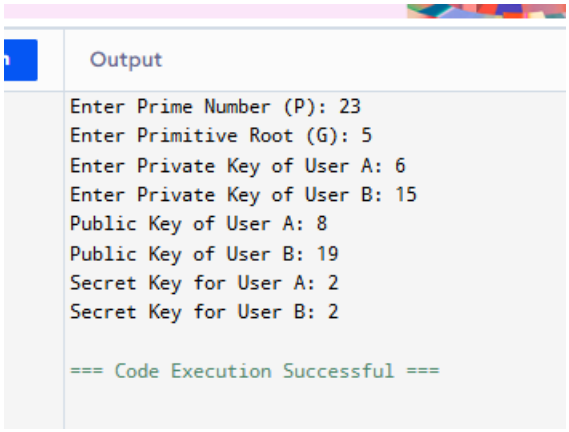
Step 10: Stop the program.

PROGRAM (Python)

main.py

```python
P = int(input("Enter Prime Number (P): "))
G = int(input("Enter Primitive Root (G): "))

a = int(input("Enter Private Key of User A: "))
b = int(input("Enter Private Key of User B: "))

A = (G ** a) % P
B = (G ** b) % P

print("Public Key of User A:", A)
print("Public Key of User B:", B)

keyA = (B ** a) % P
keyB = (A ** b) % P

print("Secret Key for User A:", keyA)
print("Secret Key for User B:", keyB)
```

OUTPUT



RESULT

Thus, the Diffie–Hellman Key Exchange Algorithm was implemented successfully, and both users generated the same shared secret key.

EXPERIMENT 7

IMPLEMENTATION OF ELLIPTIC CURVE CRYPTOGRAPHY (ECC)

AIM

To implement the Elliptic Curve Cryptography (ECC) algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the prime number (P), generator point (G), and private keys of User A and User B.

Step 3: Generate the public key of User A by multiplying the generator point with the private key.

Step 4: Generate the public key of User B by multiplying the generator point with the private key.

Step 5: Exchange the public keys between both users.

Step 6: Compute the shared secret key at User A using User B's public key.

Step 7: Compute the shared secret key at User B using User A's public key.

Step 8: Display the public keys and the shared secret keys.

Step 9: Stop the program.

PROGRAM (Python)

```python
P = int(input("Enter Prime Number (P): "))
G = int(input("Enter Generator Point (G): "))

a = int(input("Enter Private Key of User A: "))
b = int(input("Enter Private Key of User B: "))

publicA = (a * G) % P
publicB = (b * G) % P

print("Public Key of User A:", publicA)
print("Public Key of User B:", publicB)

secretA = (a * publicB) % P
secretB = (b * publicA) % P

print("Shared Secret Key (User A):", secretA)
print("Shared Secret Key (User B):", secretB)
```

OUTPUT

```
Output

Enter Prime Number (P): 23
Enter Generator Point (G): 5
Enter Private Key of User A: 6
Enter Private Key of User B: 8
Public Key of User A: 7
Public Key of User B: 17
Shared Secret Key (User A): 10
Shared Secret Key (User B): 10

=== Code Execution Successful ===
```

RESULT

Thus, the Elliptic Curve Cryptography (ECC) algorithm was implemented successfully, and both users generated the same shared secret key.

EXPERIMENT 8

IMPLEMENTATION OF BLOWFISH ALGORITHM

AIM

To implement a simplified Blowfish encryption and decryption algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext message from the user.

Step 3: Read the encryption key.

Step 4: Convert each character of the plaintext into its ASCII value.

Step 5: Encrypt each character using the given key.

Step 6: Generate the ciphertext by combining all the encrypted characters.

Step 7: Display the encrypted text.

Step 8: Apply the decryption process using the same key.

Step 9: Convert the decrypted ASCII values back to characters.

Step 10: Display the original plaintext.

Step 11: Verify that the decrypted text matches the original plaintext.

Step 12: Stop the program.

PROGRAM (Python)

main.py

```python
def encrypt(text, key):
    cipher = ""
    for ch in text:
        cipher += chr((ord(ch) + key) % 256)
    return cipher

def decrypt(cipher, key):
    plain = ""
    for ch in cipher:
        plain += chr((ord(ch) - key) % 256)
    return plain

plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))

encrypted = encrypt(plain, key)
print("Encrypted Text:", encrypted)

decrypted = decrypt(encrypted, key)
print("Decrypted Text:", decrypted)
```

OUTPUT

Output

```
Enter Plain Text: hi hello
Enter Key: 4
Encrypted Text: lm$lipps
Decrypted Text: hi hello

=== Code Execution Successful ===
```

RESULT

Thus, the Blowfish algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using the given key.

EXPERIMENT 9

IMPLEMENTATION OF RC5 ALGORITHM

AIM

To implement a simplified RC5 encryption and decryption algorithm using Python.

ALGORITHM

Step 1: Start the program.

Step 2: Read the plaintext message from the user.

Step 3: Read the encryption key.

Step 4: Convert each character of the plaintext into its ASCII value.

Step 5: Encrypt each character using the key with XOR and circular shift operations.

Step 6: Generate the ciphertext from the encrypted characters.

Step 7: Display the encrypted text.

Step 8: Decrypt the ciphertext using the same key by reversing the operations.

Step 9: Convert the decrypted values back into characters.

Step 10: Display the original plaintext.

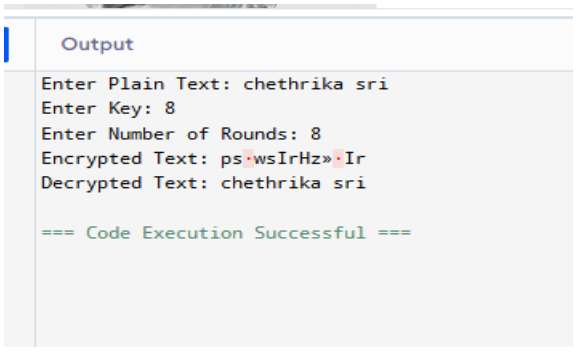
Step 11: Verify that the decrypted text matches the original plaintext.

Step 12: Stop the program.

PROGRAM (Python)

```python
def left_rotate(x):
    return ((x << 3) & 255) | (x >> 5)

def right_rotate(x):
    return (x >> 3) | ((x & 7) << 5)

def encrypt(text, key, rounds):
    cipher = ""
    for ch in text:
        c = ord(ch)
        for i in range(rounds):
            c = (c + key) % 256
            c = left_rotate(c)
            c = c ^ key
        cipher += chr(c)
    return cipher

def decrypt(cipher, key, rounds):
    plain = ""
    for ch in cipher:
        c = ord(ch)
        for i in range(rounds):
            c = c ^ key
            c = right_rotate(c)
            c = (c - key) % 256
        plain += chr(c)
    return plain

plain = input("Enter Plain Text: ")
key = int(input("Enter Key: "))
rounds = int(input("Enter Number of Rounds: "))

encrypted = encrypt(plain, key, rounds)
print("Encrypted Text:", encrypted)

decrypted = decrypt(encrypted, key, rounds)
print("Decrypted Text:", decrypted)
```

OUTPUT



RESULT

Thus, the simplified RC5 algorithm was implemented successfully, and the plaintext was encrypted and decrypted correctly using the given key and number of rounds.